of your lib folder will be available to your application when you run it. You're probably used to Ruby's auto-loading gem—Zeitwerk—but Mix handles all of this for us.

Let's execute the functions by loading a Mix-enabled IEx session. You do this by passing the -S mix flag to iex:

```
$ iex -S mix
Compiling 2 files (.ex)
iex(1)> Examples.Modules.One.hello()
:world

iex(2)> Examples.Modules.One.welcome("Steve")
"Welcome Steve"

iex(3)> Examples.Modules.One.welcome()
** (UndefinedFunctionError) function Examples.Modules.One.welcome/0
   is undefined or private. Did you mean one of:

      * welcome/1

    (examples 0.1.0) Examples.Modules.One.welcome()
```

You can easily invoke the functions defined in your module, but giving a name that doesn't exist or an incorrect number of arguments will result in an error.

Take a look at comparable code in Ruby. It's very similar, but there are significant differences:

ruby_examples/one.rb

```
class One
  def hello
    :world
  end

  def welcome(name)
    "Welcome #{name}"
  end
end

one = One.new
one.hello #> :world
one.welcome("You") #> "Welcome You"
```

Ruby methods can be class-level or object-level, and classes can be instantiated as object instances. Remember that Elixir has no objects, only functions.

This section covered one type of function (named functions), but there's another type of function that's used in different scenarios. Let's take a look at anonymous functions next.